Overview

In this assignment, you will implement the core linear data structures—**Linked Lists, Stacks, and Queues**—using C++. The primary objective is to master memory management and understand the differences between **contiguous (array-based)** and **linked (pointer-based)** memory allocation.

To simulate professional software development, you must use a **modular project structure** by separating declarations from implementations.

---

1. Project Architecture

Your project must be divided into the following file structure. **Single-file submissions will not be accepted.**

A. Common Structures

- **`Node.h`**: Contains the `struct` definition for a linked list node. This header will be shared across your Linked List, Stack, and Queue implementations.

B. Header Files (`.h`)

These files must contain the class definitions, member variables, and function prototypes. Use **include guards** to prevent compilation errors.

- **`LinkedList.h`**: Interface for the Singly Linked List.
- **`Stack.h`**: Interfaces for both `StackArray` and `StackLinkedList` classes.
- **`Queue.h`**: Interfaces for both `QueueArray` (Circular) and `QueueLinkedList` classes.

C. Implementation Files (`.cpp`)

These files contain the logic/definitions for the methods declared in the headers.

- **`LinkedList.cpp`**, **`Stack.cpp`**, **`Queue.cpp`**.

D. Driver Program

- **`main.cpp`**: Contains the `main()` function to instantiate and test each data structure.

---

2. Requirements & Functionality

Singly Linked List

- Implement `insertAtHead()`, `insertAtEnd()`, and `deleteValue()`.
- Include a `display()` function to print the list.

Stacks (LIFO)

1. **Array-Based:** Use a dynamic array. Must handle **Stack Overflow** (if capacity is reached) and **Stack Underflow**.
2. **Linked List-Based:** Must have O(1) time complexity for all operations.

- **Methods:** `push()`, `pop()`, `peek()`, `isEmpty()`.

Queues (FIFO)

1. **Circular Array-Based:** Implement a **Circular Queue** to ensure efficient use of array space. Use `front` and `rear` indices.
2. **Linked List-Based:** Maintain `front` and `rear` pointers to ensure O(1) enqueue and dequeue operations.

- **Methods:** `enqueue()`, `dequeue()`, `isFull()`, `isEmpty()`.

---

3. Code Structure Example

Header Example (`Stack.h`)

C++
```
#ifndef STACK_H
#define STACK_H

#include "Node.h"

class StackLL {
private:
    Node* top;
public:
    StackLL();
    ~StackLL(); // Must free all nodes
    void push(int value);
    int pop();
    bool isEmpty();
};

#endif
```

Source Example (`Stack.cpp`)

C++
```
#include "Stack.h"
#include <iostream>

StackLL::StackLL() : top(nullptr) {}
```

```
void StackLL::push(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = top;
    top = newNode;
}

// ... other implementations
```

---

4. Submission Guidelines

- **Memory Management:** Every `new` call must have a corresponding `delete`. Use **destructors** to clean up memory when an object goes out of scope.
- **Encapsulation:** Data members (like `top`, `front`, `rear`, `arr`) must be `private`.
- **Compilation:** Ensure your code compiles using the following command: `g++ main.cpp LinkedList.cpp Stack.cpp Queue.cpp -o dsa_test`
- **Comments:** Use meaningful comments to explain your logic, especially for circular array index wrapping.